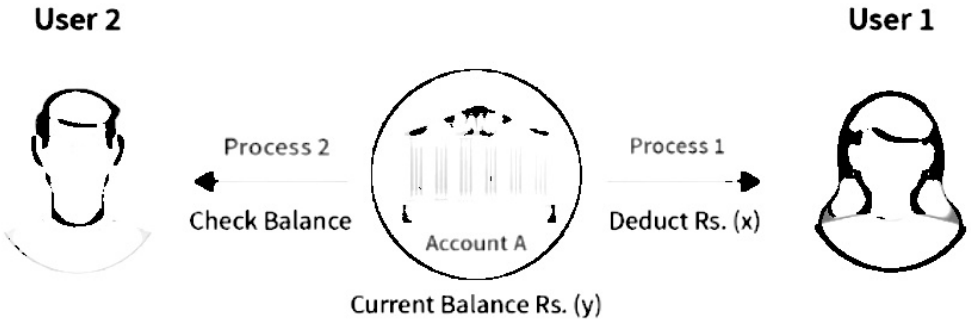
Overview

Processes Synchronization or **Synchronization** is the way by which processes that share the same memory space are managed in an operating system. It helps maintain the consistency of data by using variables or hardware so that only one process can make changes to the shared memory at a time. There are various solutions for the same such as **semaphores**, **mutex locks**, **synchronization hardware**, etc.

What is Process Synchronization in OS?

An operating system is software that manages all applications on a device and basically helps in the smooth functioning of our computer. Because of this reason, the operating system has to perform many tasks, sometimes simultaneously. This isn't usually a problem unless these simultaneously occurring processes use a common resource.

For example, consider a bank that stores the account balance of each customer in the same database. Now suppose you initially have **x** rupees in your account. Now, you take out some amount of money from your bank account, and at the same time, someone tries to look at the amount of money stored in your account. As you are taking out some money from your account, after the transaction, the total balance left will be lower than **x**. But, the transaction takes time, and hence the person reads **x** as your account balance which leads to inconsistent data. If in some way, we could make sure that only one process occurs at a time, we could ensure consistent data.



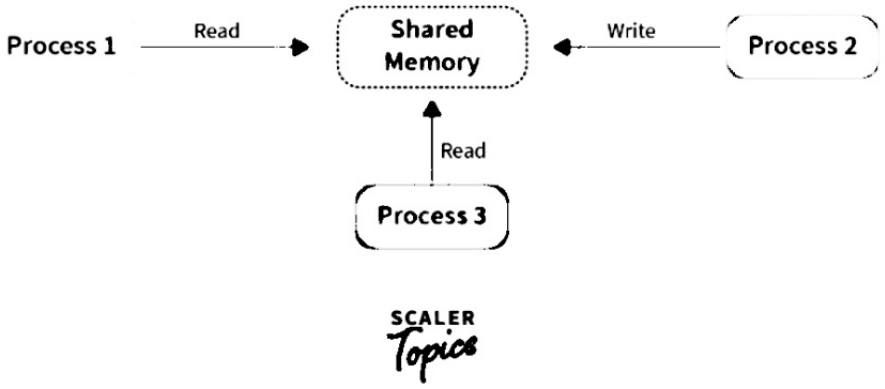
SCALER Topics

In the above image, if **Process1** and **Process2** happen at the same time, user 2 will get the wrong account balance as Y because of **Process1** being transacted when the balance is **X**.

Inconsistency of data can occur when various processes share a common resource in a system which is why there is a need for process synchronization in the operating system.

How Process Synchronization in OS Works?

Let us take a look at why exactly we need Process Synchronization. For example, If a **process1** is trying to read the data present in a memory location while another **process2** is trying to change the data present at the same location, there is a high chance that the data read by the **process1** will be incorrect.



Let us look at different elements/sections of a program:

- **Entry Section:** The entry Section decides the entry of a process.
- **Critical Section:** The Critical section allows and makes sure that only one process is modifying the shared data.

The Critical-Section Problem

Consider a system consisting of n processes $\{P_0, P_1, ..., P_{n-1}\}$. Each process has a segment of code, called a critical section, in which the process may be changing common variables, updating a table, writing a file, and so on. The important feature of the system is that, when one process is executing in its critical section, no other process is to be allowed to execute in its critical section. That is, no two processes are executing in their critical sections at the same time. The *critical-section problem* is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section. The section of code implementing this request is the entry section. The critical section may be followed by an exit section. The remaining code is the remainder section. The general structure of a typical process P_i is shown in

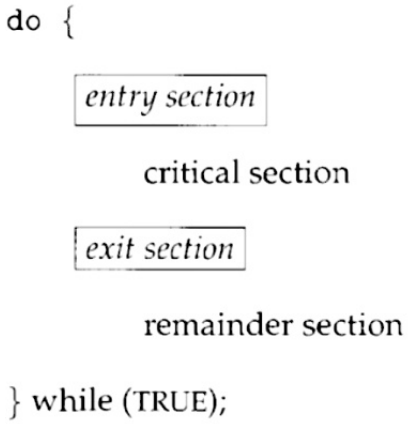


Figure 6.1 General structure of a typical process P_i.

Figure 6.1. The entry section and exit section are enclosed in boxes to highlight these important segments of code.

A solution to the critical-section problem must satisfy the following three requirements:

1. **Mutual exclusion**. If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress**. If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.
3. **Bounded waiting**. There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

We assume that each process is executing at a nonzero speed. However, we can make no assumption concerning the relative speed of the n processes.

At a given point in time, many kernel-mode processes may be active in the operating system. As a result, the code implementing an operating system (*kernel code*) is subject to several possible race conditions. Consider as an example a kernel data structure that maintains a list of all open files in the system. This list must be modified when a new file is opened or closed (adding the file to the list or removing it from the list). If two processes were to open files simultaneously, the separate updates to this list could result in a race condition. Other kernel data structures that are prone to possible race conditions include structures for maintaining memory allocation, for maintaining process lists, and for interrupt handling. It is up to kernel developers to ensure that the operating system is free from such race conditions.

Two general approaches are used to handle critical sections in operating systems: (1) **preemptive kernels** and (2) **nonpreemptive kernels**. A preemptive kernel allows a process to be preempted while it is running in kernel mode. A nonpreemptive kernel does not allow a process running in kernel mode

to be preempted; a kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU. Obviously, a nonpreemptive kernel is essentially free from race conditions on kernel data structures, as only one process is active in the kernel at a time. We cannot say the same about preemptive kernels, so they must be carefully designed to ensure that shared kernel data are free from race conditions. Preemptive kernels are especially difficult to design for SMP architectures, since in these environments it is possible for two kernel-mode processes to run simultaneously on different processors.

Why, then, would anyone favor a preemptive kernel over a nonpreemptive one? A preemptive kernel is more suitable for real-time programming, as it will allow a real-time process to preempt a process currently running in the kernel. Furthermore, a preemptive kernel may be more responsive, since there is less risk that a kernel-mode process will run for an arbitrarily long period before relinquishing the processor to waiting processes. Of course, this effect can be minimized by designing kernel code that does not behave in this way. Later in this chapter, we explore how various operating systems manage preemption within the kernel.

Peterson's Solution

Next, we illustrate a classic software-based solution to the critical-section problem known as **Peterson's solution**. Because of the way modern computer architectures perform basic machine-language instructions, such as `load` and `store`, there are no guarantees that Peterson's solution will work correctly on such architectures. However, we present the solution because it provides a good algorithmic description of solving the critical-section problem and illustrates some of the complexities involved in designing software that addresses the requirements of mutual exclusion, progress, and bounded waiting.

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections. The processes are numbered P_0 and P_1. For convenience, when presenting P_i, we use P_j to denote the other process; that is, `j` equals $1 - $ `i`.

Peterson's solution requires the two processes to share two data items:

```
int turn;
boolean flag[2];
```

The variable `turn` indicates whose turn it is to enter its critical section. That is, if `turn == i`, then process P_i is allowed to execute in its critical section. The `flag` array is used to indicate if a process *is ready* to enter its critical section. For example, if `flag[i]` is `true`, this value indicates that P_i is ready to enter its critical section. With an explanation of these data structures complete, we are now ready to describe the algorithm shown in Figure 6.2.

To enter the critical section, process P_i first sets `flag[i]` to be `true` and then sets `turn` to the value `j`, thereby asserting that if the other process wishes to enter the critical section, it can do so. If both processes try to enter at the same time, `turn` will be set to both `i` and `j` at roughly the same time. Only one of these assignments will last; the other will occur but will be overwritten immediately.

```
do {
    flag[i] = TRUE;
    turn = j;
    while (flag[j] && turn == j);

        critical section

    flag[i] = FALSE;

        remainder section

} while (TRUE);
```

Figure 6.2 The structure of process P_i in Peterson's solution.

The eventual value of `turn` determines which of the two processes is allowed to enter its critical section first.

We now prove that this solution is correct. We need to show that:

1. Mutual exclusion is preserved.
2. The progress requirement is satisfied.
3. The bounded-waiting requirement is met.

To prove property 1, we note that each P_i enters its critical section only if either `flag[j] == false` or `turn == i`. Also note that, if both processes can be executing in their critical sections at the same time, then `flag[0] == flag[1] == true`. These two observations imply that P_0 and P_1 could not have successfully executed their `while` statements at about the same time, since the value of `turn` can be either 0 or 1 but cannot be both. Hence, one of the processes —say, P_j—must have successfully executed the while statement, whereas P_i had to execute at least one additional statement ("`turn == j`"). However, at that time, `flag[j] == true` and `turn == j`, and this condition will persist as long as P_j is in its critical section; as a result, mutual exclusion is preserved.

To prove properties 2 and 3, we note that a process P_i can be prevented from entering the critical section only if it is stuck in the while loop with the condition `flag[j] == true` and `turn == j`; this loop is the only one possible. If P_j is not ready to enter the critical section, then `flag[j] == false`, and P_i can enter its critical section. If P_j has set `flag[j]` to `true` and is also executing in its `while` statement, then either `turn == i` or `turn == j`. If `turn == i`, then P_i will enter the critical section. If `turn == j`, then P_j will enter the critical section. However, once P_j exits its critical section, it will reset `flag[j]` to `false`, allowing P_i to enter its critical section. If P_j resets `flag[j]` to `true`, it must also set `turn` to `i`. Thus, since P_i does not change the value of the variable `turn` while executing the `while` statement, P_i will enter the critical section (progress) after at most one entry by P_j (bounded waiting).

```
do {

    acquire lock

        critical section

    release lock

        remainder section

} while (TRUE);
```

Figure 6.3 Solution to the critical-section problem using locks.

Synchronization Hardware

We have just described one software-based solution to the critical-section problem. However, as mentioned, software-based solutions such as Peterson's are not guaranteed to work on modern computer architectures. Instead, we can generally state that any solution to the critical-section problem requires a simple tool—a **lock**. Race conditions are prevented by requiring that critical regions be protected by locks. That is, a process must acquire a lock before entering a critical section; it releases the lock when it exits the critical section. This is illustrated in Figure 6.3.

In the following discussions, we explore several more solutions to the critical-section problem using techniques ranging from hardware to software-based APIs available to application programmers. All these solutions are based on the premise of locking; however, as we shall see, the designs of such locks can be quite sophisticated.

We start by presenting some simple hardware instructions that are available on many systems and showing how they can be used effectively in solving the critical-section problem. Hardware features can make any programming task easier and improve system efficiency.

The critical-section problem could be solved simply in a uniprocessor environment if we could prevent interrupts from occurring while a shared variable was being modified. In this manner, we could be sure that the current sequence of instructions would be allowed to execute in order without preemption. No other instructions would be run, so no unexpected modifications could be made to the shared variable. This is often the approach taken by nonpreemptive kernels.

Unfortunately, this solution is not as feasible in a multiprocessor environment. Disabling interrupts on a multiprocessor can be time consuming, as the

```
boolean TestAndSet(boolean *target) {
   boolean rv = *target;
   *target = TRUE;
   return rv;
}
```

Figure 6.4 The definition of the `TestAndSet()` instruction.

```
do {
   while (TestAndSet(&lock))
      ; // do nothing

      // critical section

   lock = FALSE;

      // remainder section
} while (TRUE);
```

Figure 6.5 Mutual-exclusion implementation with `TestAndSet()`.

message is passed to all the processors. This message passing delays entry into each critical section, and system efficiency decreases. Also consider the effect on a system's clock if the clock is kept updated by interrupts.

Many modern computer systems therefore provide special hardware instructions that allow us either to test and modify the content of a word or to swap the contents of two words atomically—that is, as one uninterruptible unit. We can use these special instructions to solve the critical-section problem in a relatively simple manner. Rather than discussing one specific instruction for one specific machine, we abstract the main concepts behind these types of instructions by describing the `TestAndSet()` and `Swap()` instructions.

The `TestAndSet()` instruction can be defined as shown in Figure 6.4. The important characteristic of this instruction is that it is executed atomically. Thus, if two `TestAndSet()` instructions are executed simultaneously (each on a different CPU), they will be executed sequentially in some arbitrary order. If the machine supports the `TestAndSet()` instruction, then we can implement mutual exclusion by declaring a Boolean variable `lock`, initialized to `false`. The structure of process P_i is shown in Figure 6.5.

The `Swap()` instruction, in contrast to the `TestAndSet()` instruction, operates on the contents of two words; it is defined as shown in Figure 6.6. Like the `TestAndSet()` instruction, it is executed atomically. If the machine supports the `Swap()` instruction, then mutual exclusion can be provided as follows. A global Boolean variable `lock` is declared and is initialized to `false`. In addition, each process has a local Boolean variable `key`. The structure of process P_i is shown in Figure 6.7.

Although these algorithms satisfy the mutual-exclusion requirement, they do not satisfy the bounded-waiting requirement. In Figure 6.8, we present another algorithm using the `TestAndSet()` instruction that satisfies all the critical-section requirements. The common data structures are

```
void Swap(boolean *a, boolean *b) {
   boolean temp = *a;
   *a = *b;
   *b = temp;
}
```

Figure 6.6 The definition of the `Swap()` instruction.

```
do {
   key = TRUE;
   while (key == TRUE)
      Swap(&lock, &key);

      // critical section

   lock = FALSE;

      // remainder section
} while (TRUE);
```

Figure 6.7 Mutual-exclusion implementation with the `Swap()` instruction.

```
boolean waiting[n];
boolean lock;
```

These data structures are initialized to `false`. To prove that the mutual-exclusion requirement is met, we note that process P_i can enter its critical section only if either `waiting[i] == false` or `key == false`. The value of `key` can become `false` only if the `TestAndSet()` is executed. The first process to execute the `TestAndSet()` will find `key == false`; all others must wait. The variable `waiting[i]` can become `false` only if another process leaves its critical section; only one `waiting[i]` is set to `false`, maintaining the mutual-exclusion requirement.

```
do {
   waiting[i] = TRUE;
   key = TRUE;
   while (waiting[i] && key)
      key = TestAndSet(&lock);
   waiting[i] = FALSE;

      // critical section

   j = (i + 1) % n;
   while ((j != i) && !waiting[j])
      j = (j + 1) % n;

   if (j == i)
      lock = FALSE;
   else
      waiting[j] = FALSE;

      // remainder section
} while (TRUE);
```

Figure 6.8 Bounded-waiting mutual exclusion with `TestAndSet()`.

To prove that the progress requirement is met, we note that the arguments presented for mutual exclusion also apply here, since a process exiting the critical section either sets `lock` to `false` or sets `waiting[j]` to `false`. Both allow a process that is waiting to enter its critical section to proceed.

To prove that the bounded-waiting requirement is met, we note that, when a process leaves its critical section, it scans the array `waiting` in the cyclic ordering $(i + 1, i + 2, ..., n - 1, 0, ..., i - 1)$. It designates the first process in this ordering that is in the entry section (`waiting[j] == true`) as the next one to enter the critical section. Any process waiting to enter its critical section will thus do so within $n - 1$ turns.

Unfortunately for hardware designers, implementing atomic `TestAndSet()` instructions on multiprocessors is not a trivial task. Such implementations are discussed in books on computer architecture.

Semaphores

The hardware-based solutions to the critical-section problem presented in Section 6.4 are complicated for application programmers to use. To overcome this difficulty, we can use a synchronization tool called a semaphore.

A semaphore `S` is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: `wait()` and `signal()`. The `wait()` operation was originally termed `P` (from the Dutch *proberen*, "to test"); `signal()` was originally called `V` (from *verhogen*, "to increment"). The definition of `wait()` is as follows:

```
wait(S) {
    while S <= 0
       ; // no-op
    S--;
}
```

The definition of `signal()` is as follows:

```
signal(S) {
    S++;
}
```

All modifications to the integer value of the semaphore in the `wait()` and `signal()` operations must be executed indivisibly. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value. In addition, in the case of `wait(S)`, the testing of the integer value of `S` ($S \leq 0$), as well as its possible modification (`S--`), must be executed without interruption. We shall see how these operations can be implemented in Section 6.5.2; first, let us see how semaphores can be used.

6.5.1 Usage

Operating systems often distinguish between counting and binary semaphores. The value of a **counting semaphore** can range over an unrestricted domain. The value of a **binary semaphore** can range only between 0 and 1. On some

systems, binary semaphores are known as **mutex locks**, as they are locks that provide *mut*ual *ex*clusion.

We can use binary semaphores to deal with the critical-section problem for multiple processes. The n processes share a semaphore, `mutex`, initialized to 1. Each process P_i is organized as shown in Figure 6.9.

Counting semaphores can be used to control access to a given resource consisting of a finite number of instances. The semaphore is initialized to the number of resources available. Each process that wishes to use a resource performs a `wait()` operation on the semaphore (thereby decrementing the count). When a process releases a resource, it performs a `signal()` operation (incrementing the count). When the count for the semaphore goes to 0, all resources are being used. After that, processes that wish to use a resource will block until the count becomes greater than 0.

We can also use semaphores to solve various synchronization problems. For example, consider two concurrently running processes: P_1 with a statement S_1 and P_2 with a statement S_2. Suppose we require that S_2 be executed only after S_1 has completed. We can implement this scheme readily by letting P_1 and P_2 share a common semaphore `synch`, initialized to 0, and by inserting the statements

```
S1;
signal(synch);
```

in process P_1 and the statements

```
wait(synch);
S2;
```

in process P_2. Because `synch` is initialized to 0, P_2 will execute S_2 only after P_1 has invoked `signal(synch)`, which is after statement S_1 has been executed.

6.5.2 Implementation

The main disadvantage of the semaphore definition given here is that it requires busy waiting. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. This continual looping is clearly a problem in a real multiprogramming system,

```
do {
   wait(mutex);

      // critical section

   signal(mutex);

      // remainder section
} while (TRUE);
```

Figure 6.9 Mutual-exclusion implementation with semaphores.

6.6.3 The Dining-Philosophers Problem

Consider five philosophers who spend their lives thinking and eating. The philosophers share a circular table surrounded by five chairs, each belonging

```
do {
   wait(mutex);
   readcount++;
   if (readcount == 1)
      wait(wrt);
   signal(mutex);
      . . .
   // reading is performed
      . . .
   wait(mutex);
   readcount--;
   if (readcount == 0)
      signal(wrt);
   signal(mutex);
} while (TRUE);
```

Figure 6.13 The structure of a reader process.

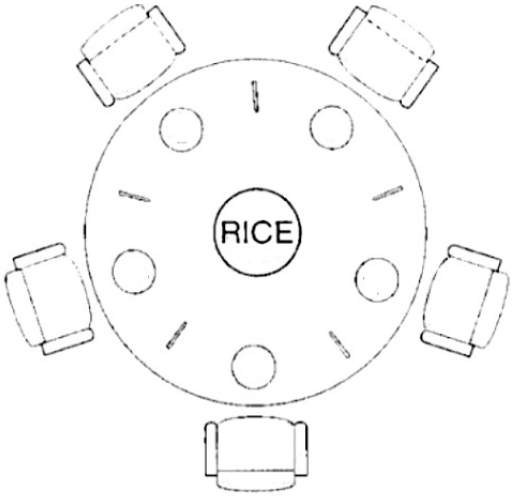


Figure 6.14 The situation of the dining philosophers.

to one philosopher. In the center of the table is a bowl of rice, and the table is laid with five single chopsticks (Figure 6.14). When a philosopher thinks, she does not interact with her colleagues. From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her (the chopsticks that are between her and her left and right neighbors). A philosopher may pick up only one chopstick at a time. Obviously, she cannot pick up a chopstick that is already in the hand of a neighbor. When a hungry philosopher has both her chopsticks at the same time, she eats without releasing her chopsticks. When she is finished eating, she puts down both of her chopsticks and starts thinking again.

The *dining-philosophers problem* is considered a classic synchronization problem neither because of its practical importance nor because computer scientists dislike philosophers but because it is an example of a large class of concurrency-control problems. It is a simple representation of the need to allocate several resources among several processes in a deadlock-free and starvation-free manner.

One simple solution is to represent each chopstick with a semaphore. A philosopher tries to grab a chopstick by executing a `wait()` operation on that semaphore; she releases her chopsticks by executing the `signal()` operation on the appropriate semaphores. Thus, the shared data are

```
semaphore chopstick[5];
```

where all the elements of `chopstick` are initialized to 1. The structure of philosopher *i* is shown in Figure 6.15.

Although this solution guarantees that no two neighbors are eating simultaneously, it nevertheless must be rejected because it could create a deadlock. Suppose that all five philosophers become hungry simultaneously and each grabs her left chopstick. All the elements of `chopstick` will now be equal to 0. When each philosopher tries to grab her right chopstick, she will be delayed forever.

Several possible remedies to the deadlock problem are listed next.

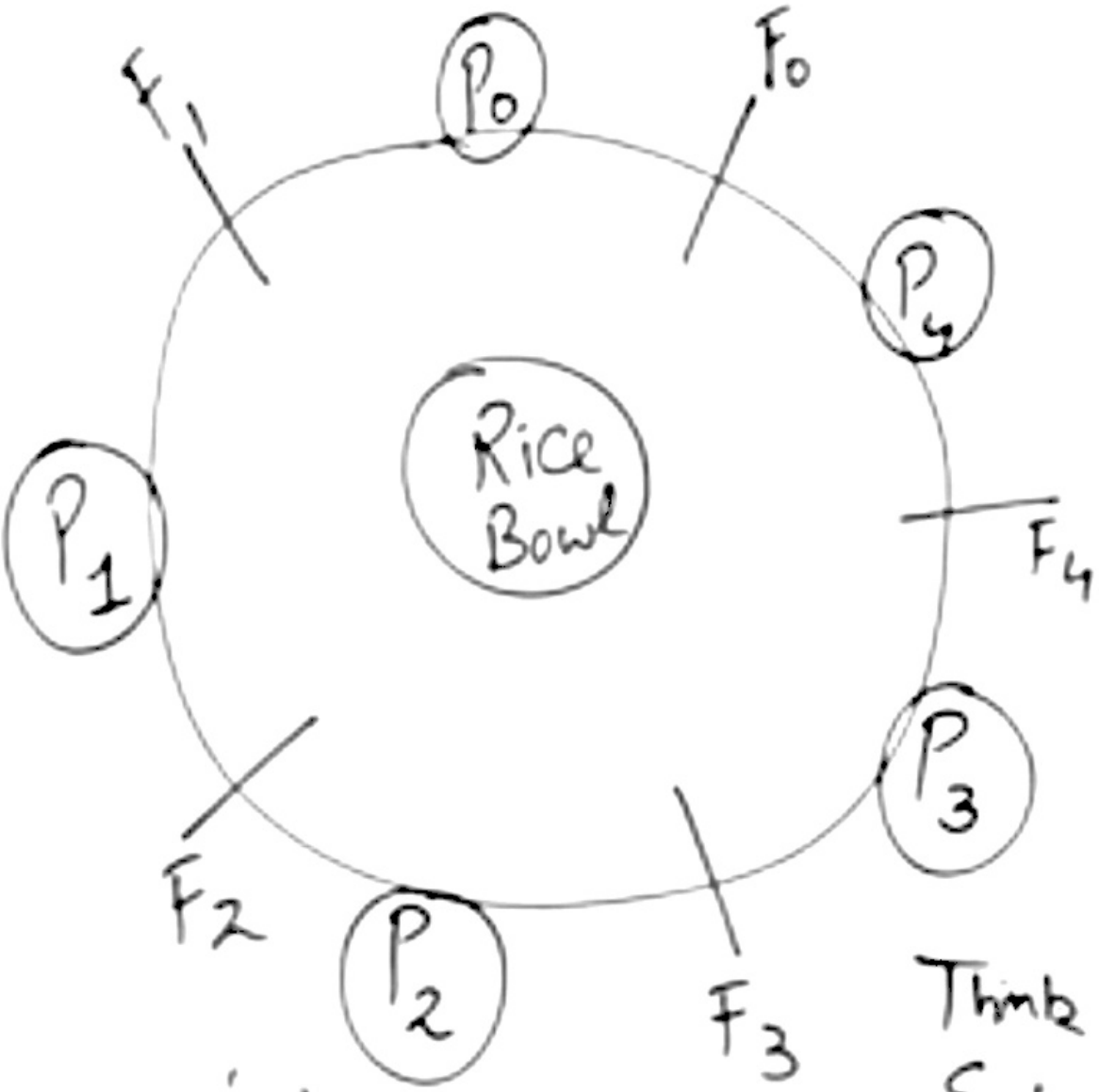
- Allow at most four philosophers to be sitting simultaneously at the table.

```
do {
   wait(chopstick[i]);
   wait(chopstick[(i+1) % 5]);
      . . .
   // eat
      . . .
   signal(chopstick[i]);
   signal(chopstick[(i+1) % 5]);
      . . .
   // think
      . . .
} while (TRUE);
```

Figure 6.15 The structure of philosopher *i*.

- Allow a philosopher to pick up her chopsticks only if both chopsticks are available (to do this, she must pick them up in a critical section).
- Use an asymmetric solution; that is, an odd philosopher picks up first her left chopstick and then her right chopstick, whereas an even philosopher picks up her right chopstick and then her left chopstick.

In Section 6.7, we present a solution to the dining-philosophers problem that ensures freedom from deadlocks. Note, however, that any satisfactory solution to the dining-philosophers problem must guard against the possibility that one of the philosophers will starve to death. A deadlock-free solution does not necessarily eliminate the possibility of starvation.



'5' Philosophers
'5' Forks

Think
Eat

```
Void philosopher (void)
{
  while (true)
  {
    Thinking(),
    take_fork (i), ← Left fork
    take_fork((i+1) % N); ← Right fork
    EAT(),
    Put fork(i), f0
    Put fork((i+1) % N), f1
  }
}
```

Call I. P0 P1

f0 f1

f(i+1) mod 5

i mod 5

N = No of forks

```
Void philosopher (void)
{
  while (true)
  {
    Thinking(),
    take_fork (i), ← Left fork
    take_fork((i+1) % N), ← Right fork
    EAT(),
    Put fork(i), f0 f1
    Put fork((i+1) % N), f1 f2
  }
}
```

Call I

P0 P1

f0 f1 f1 f2

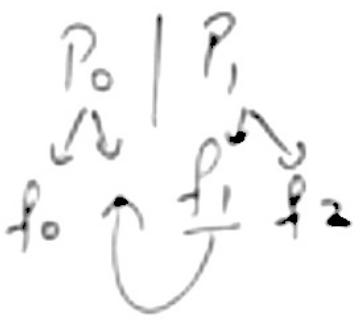
f(0+1) mod 5

1 mod 5

N = No of forks

```
Void philosopher (void)
{
  while (true)
  {
    Thinking(),
    take_fork (i),
    take_fork((i+1) % N),
    EAT(),
    Put forks(i),
    Put forks((i+1) % N),
  }
}
```

Case 2



```
Void philosopher (void)
{
  while (true)
  {
    Thinking(),
  Wait ( take_forks (Si))
  Wait ( take_forks (S((i+1) mod N))
    EAT(),
  Signal(Put forks (i),)
  Signal(Put forks ((i+1) % N),
  }
}
```

(4+1) mod 5
5 mod 5
= 0

S[i] five Semaphores

S_0 S_1 S_2 S_3 S_4
↓ ↓ ↓ ↓ ↓
1 1 1 1 1

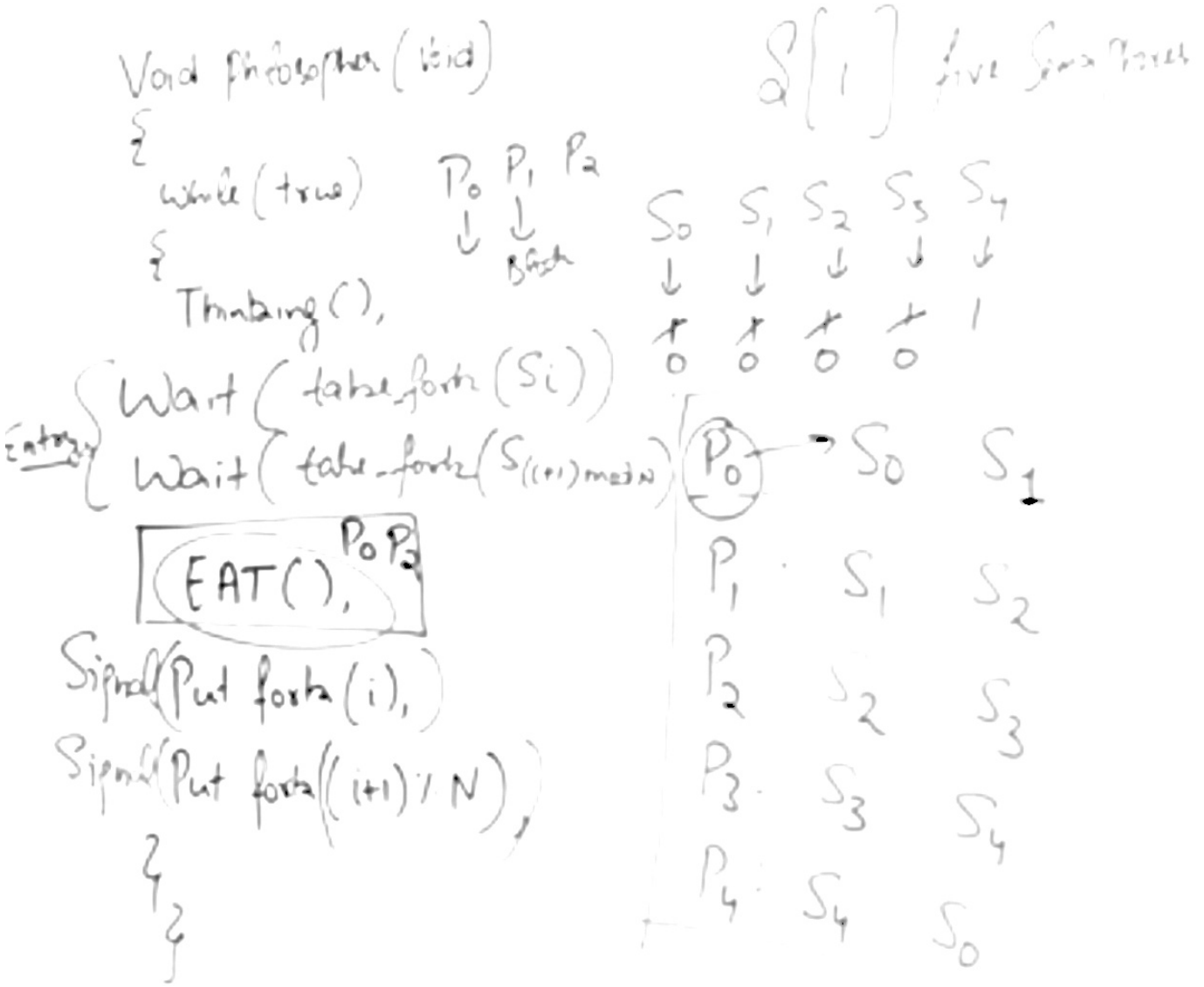
P_0 → S_0 S_1
P_1 : S_1 S_2
P_2 : S_2 S_3
P_3 : S_3 S_4
P_4 : S_4 S_0

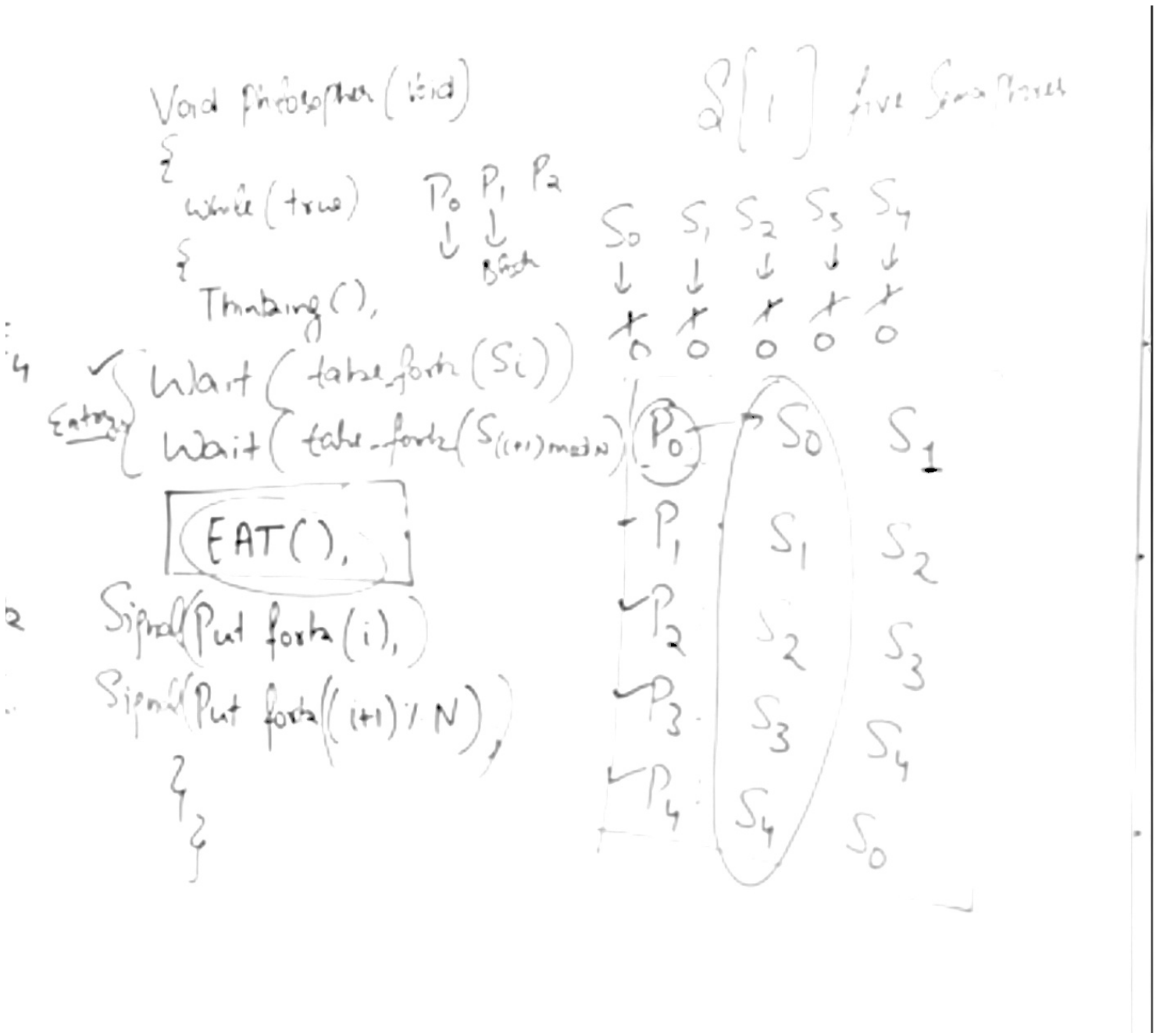
```
Void philosopher (void)
{
  while (true)
  {
    Thinking(),
    Wait (take_fork (Si))
    Wait (take_fork (S((i+1) mod N))
    EAT(),
    Signal(Put fork (i),
    Signal(Put fork((i+1) % N),
  }
}
```

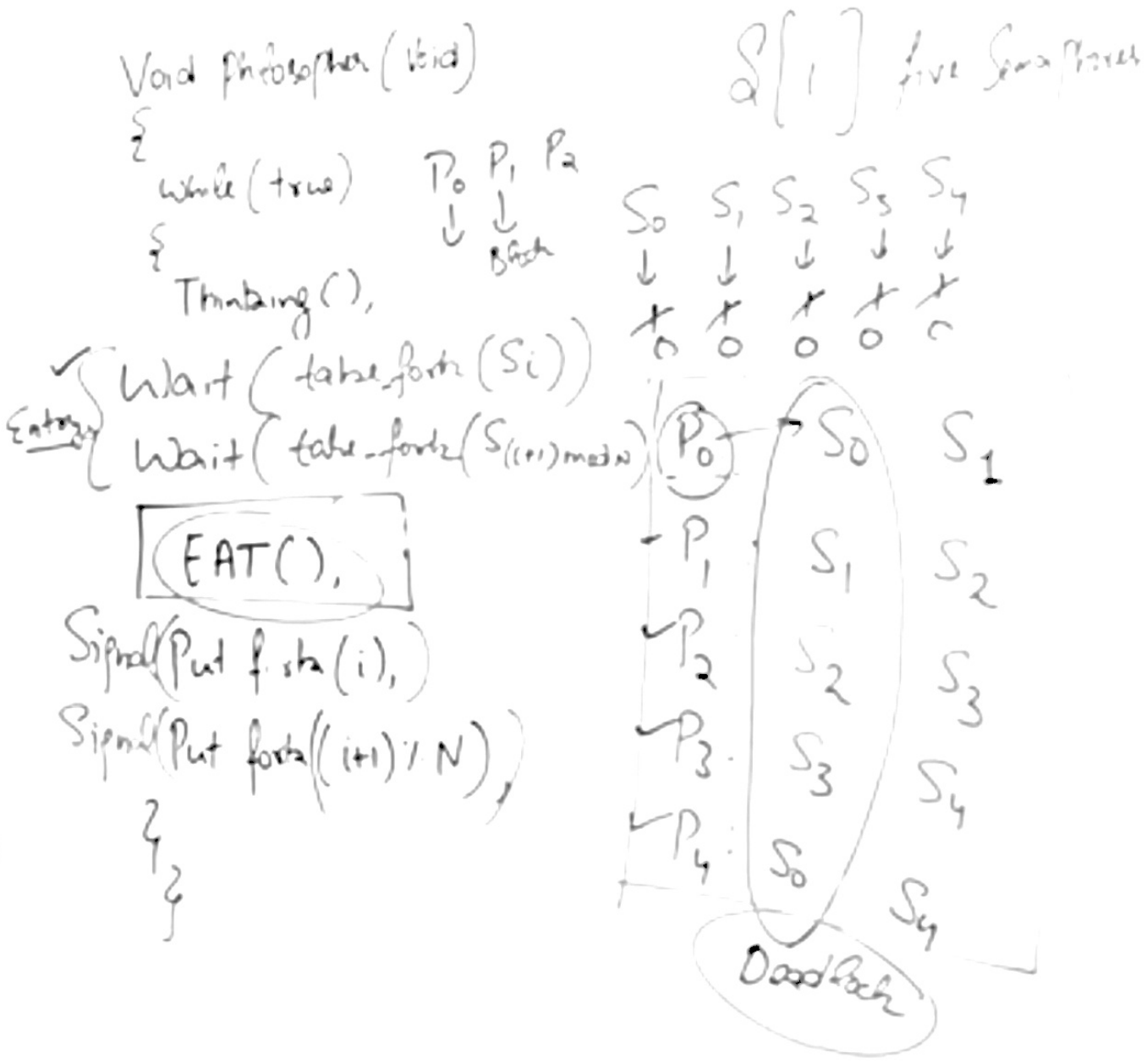
S[i] five Semaphores

S_0	S_1	S_2	S_3	S_4
↓	↓	↓	↓	↓
1	1	1	1	1

P_0	S_0	S_1
P_1	S_1	S_2
P_2	S_2	S_3
P_3	S_3	S_4
P_4	S_4	S_0

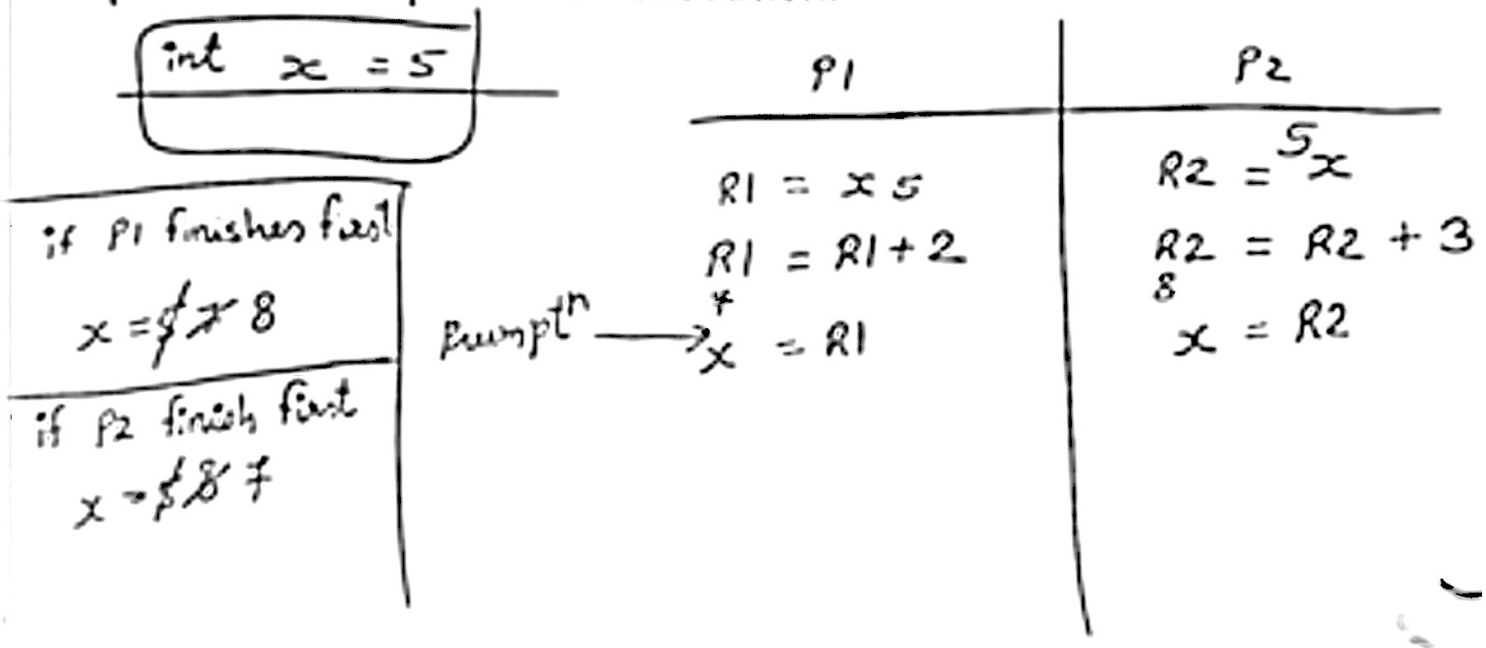

Void philosopher (void)
{
while (true)
{
Thinking(),
Wait (take_fork1 (Si))
Wait (take_fork2 (S((i+1) mod N))
EAT(),
Signal(Put fork1 (i),
Signal(Put fork2 ((i+1) % N),
}
}
Critical
P0 P1 P2
Both
S[1] five Semaphores
S0 S1 S2 S3 S4
1 1 1 1 1
0 0 0 0
P0 → S0 S1
P1 S1 S2
P2 S2 S3
P3 S3 S4
P4 S4 S0


Void philosopher (void)
{
while (true)
{
Thinking(),
Wait (take_fork (Si))
Wait (take_fork (S((i+1) mod N))
EAT(),
Signal(Put forks(i),
Signal(Put forks((i+1) % N),
}
}
S[1] five Semaphores
P0 P1 P2
Block
S0 S1 S2 S3 S4
1 1 1 1 1
0 0 0 0 0
P0 S0 S1
P1 S1 S2
P2 S2 S3
P3 S3 S4
P4 S4 S0


Void philosopher (void)
{
while (true)
{
Thinking(),
Wait (take_fork (Si))
Wait (take_fork ((i+1) mod N))
EAT(),
Signal(Put fork(i),
Signal(Put fork((i+1) % N),
}
}
Entry
S[1] five Semaphores
P0 P1 P2
Block
S0 S1 S2 S3 S4
1 1 1 1 1
0 0 0 0 0
P0 S0 S1
P1 S1 S2
P2 S2 S3
P3 S3 S4
P4 S0 S4
Deadlock

Race Condition

A race condition is an undesirable situation, it occurs when the fi result of concurrent processes depends on the sequence in which t processes complete their execution.

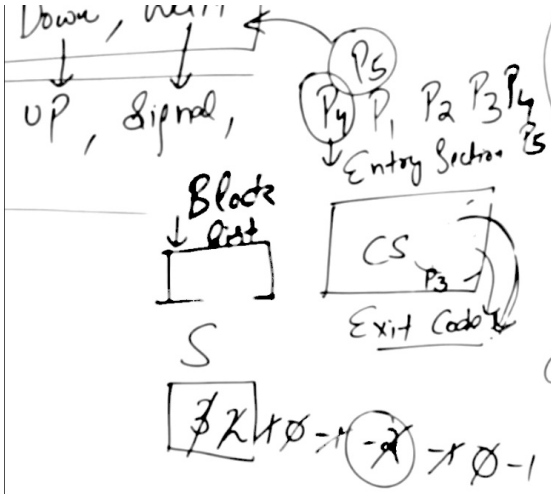


P, wait

```
Down( Semaphore S)
{
  S.Value = S.Value - 1;
  if ( S.Value < 0)
    {
      Put Process (PCB) in
      Suspended list, Sleep();
    }
  else
    return;
}
```

V, Signal, Post

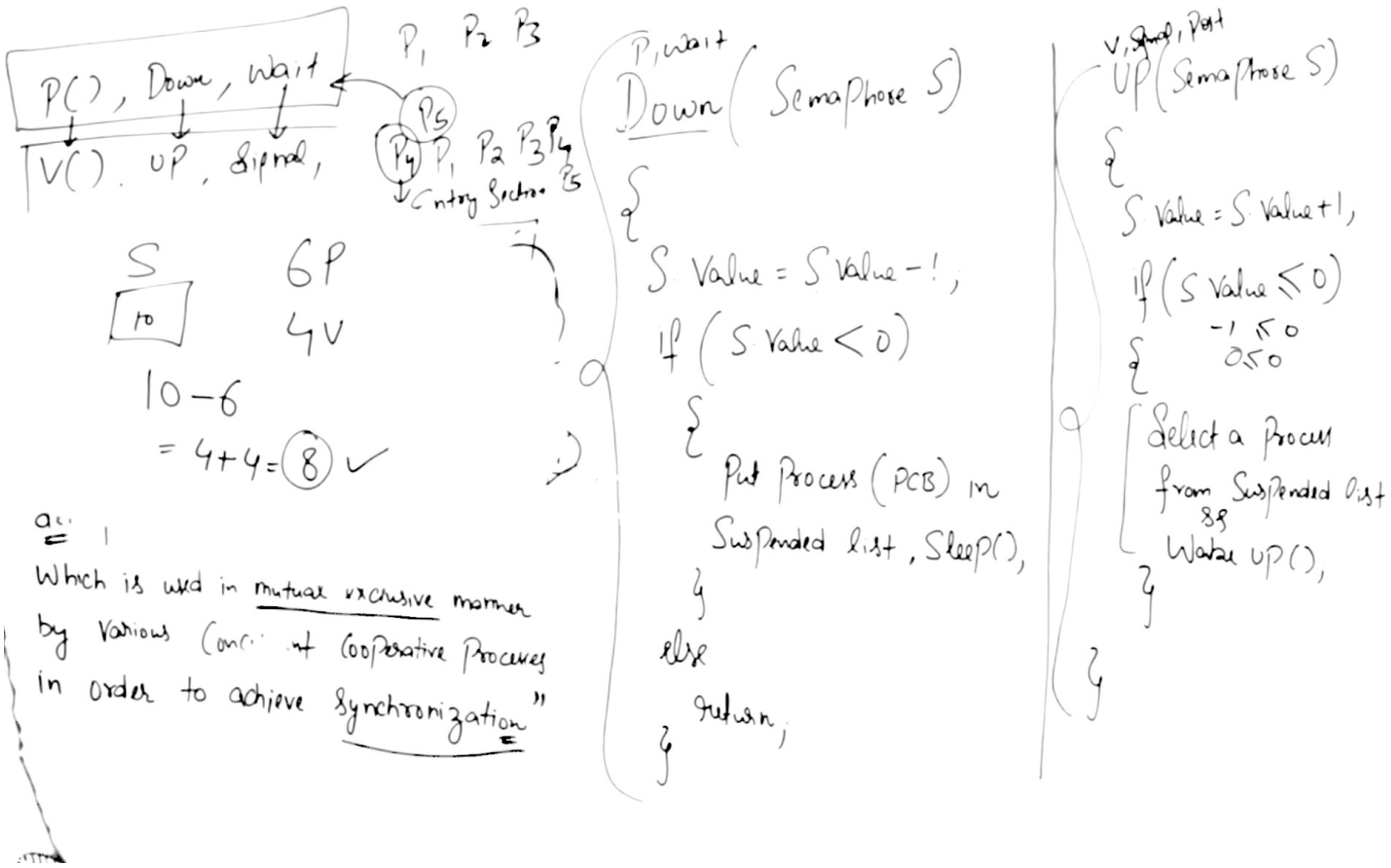
```
Up(Semaphore S)
{
  S.Value = S.Value + 1;
  if (S.Value <= 0)
  {
    Select a Process
    from Suspended list
    Wake up();
  }
}
```



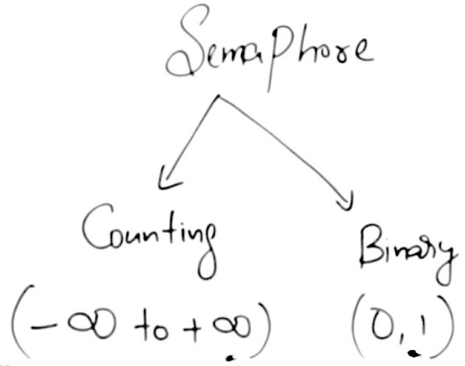
s used in mutual exclusive manner
ious Concurrent Cooperative Processes
er to achieve Synchronization"

```
Down( Semaphore S)
{
  S.Value = S.Value - 1;
  if ( S.Value < 0)
    {
      Put process (PCB) in
      Suspended list, Sleep();
    }
  else
    return;
}
```

```
Up( Semaphore S)
{
  S.Value = S.Value + 1;
  if (S.Value <= 0)
      -1 <= 0
      0 <= 0
  {
    Select a Process
    from Suspended list
    &&
    Wake up();
  }
}
```



Which is used in mutual exclusive manner by various Concurrent Cooperative Processes in order to achieve Synchronization"



"Semaphore is an integer variable which is used in mutual exclusive manner by various Concurrent Cooperative Process in order to achieve Synchronization"

```
Down( Semaphore S)
{
  S.Value = S.Value - 1;
  if ( S.Value < 0)
    {
      Put Process (PCB) in
      Suspended list Sleep();
    }
  else
    return;
}
```

```
Up(Sem
{
  S.Value
  if (S.
  {
    Select
    from
      Wak
  }
}
```

Solution of Sleeping Barber Problem using Semaphores

Chairs = n

Semaphore customer = 0 — No. of customers in waiting room

Semaphore barber = 0 — Barber is idle (sleeping)

Semaphore mutex = 1 — (mutual Exclusion)

int waiting = 0 — No. of waiting customer

Customer Process	Barber Process

while (true)
{ wait (mutex); 0
if (waiting < chairs)
{ 1 < n-1
waiting = waiting + 1; 2
signal (customer); 2
signal (mutex); 1
wait (barber); 0
get_haircut();
}
else
signal (mutex);
	while (true)
	{ wait (customer); 1
	wait (mutex); 0
	waiting = waiting - 1; 1
	signal (barber); 1
	signal (mutex); 1
	cut_hair();
	}

① There is one barber, one barber chair and n waiting chairs.

② If there is no customer, barber sleeps in his own chair.

③ When customer arrives, he has to wake up barber.

④ When many customers come and waiting chairs are empty, they sit on waiting chairs else they leave if no chair is empty.

⑤

(i) Semaphore customer counts waiting customers (excluding the customer in the barber chair, who is not waiting).

(ii) Semaphore barber counts the no of idle/active (0/1) barber.

(iii) Semaphore mutex used for mutual Exclusion

⑥ Integer variable waiting also counts the no. of waiting customers.

The reason for using variable waiting is that there is no way to read the current value of semaphore customer.

swat